IP Camera Module — How We Built It, Step by Step

Speaker script · Final presentation Part 01 · slides 05–15

ZHU Kecheng · SD5531 Team 02 · 6 November 2026

> About 6½ minutes at a normal pace (≈1,050 words). The story follows the order in which we actually worked: almost every design choice came from a test that went wrong.
>
> Text in ***[brackets]*** is a cue (a slide or a gesture); do not read it aloud.
>
> One change of order: we built the tracker (slides 13–14) just before the gecko trigger (slides 11–12). The script keeps the deck order and says so in one sentence.
>
> Every test used a green toy lizard, not a real gecko.

Timeline

When	Step	What we learned
30 Oct	Wiring; first 60 s recordings at home and at school	Camera, PoE and ROCK 4C+ work together; 1,200 frames decoded, copy checked with SHA-256
5 Nov 02:51	Latency test with a clock on the Mac screen	0.23 s camera → ROCK; starting after a trigger loses ≥ 0.4 s
03:07	RAM ring buffer + manual trigger	Pre-roll works; a timing bug was found and fixed
03:24	Frame-difference detection, hand at home	4 events, each with 3.7–4.65 s before the hand
12:23–14:56	First long run at school	86 recordings in 2.5 h, none of a gecko
15:44	Detection area + scene rule + 0.5 s persistence	0 triggers from outside; a hand inside still records
16:10	Threshold 25 → 15	The moving toy lizard is now caught
16:22 / 16:47	Tracker: brightness only, then colour	Box on the tail only → box on the whole lizard
17:47 / 18:04 / 18:45	Gecko trigger, three rounds	3 → 2 → 0 hand-only clips

Script

Slide 05 IP Camera Module ≈20 s

I'm Kecheng, and I built the video part. Instead of only listing what the system does, I'll tell you how we got there — in the order we did it, because almost every design choice came from a test that went wrong.

Slide 06 Step 1 · Get a picture ≈35 s

Step one, on the 30th of October, was simply to get a picture. We connected the camera to a PoE injector, which sends 48 volts and video over one network cable, and joined the injector, the ROCK 4C+ and the MacBook on a gigabit switch. The ROCK recorded 60 seconds from the camera — 1,200 frames at 2560 by 1440. We decoded every frame and checked the copy on the Mac with SHA-256.

One detail on this slide came back to us later. ***[Point to the right USB-C]*** The ROCK is powered by the Mac's USB-C port. It stopped suddenly twice at school, and this power path is our first suspect — not yet confirmed.

Slide 07 Step 2 · Measure before designing ≈40 s

Step two, on the 5th of November, was to measure before designing. The camera sends two streams: a full-resolution main stream for recording, and a small substream that is cheap to analyse.

To measure delay, we pointed the camera at a millisecond clock on the Mac screen. ***[Point to the clock photos]*** A frame reaches the ROCK 0.23 seconds after it is filmed. Then we tried the obvious design: start FFmpeg only when something moves. Six cold starts showed that the first frame is always about 0.4 seconds late, and FFmpeg needs about 2 seconds before it writes anything. So "start after motion" always loses the beginning — the very moment a student wants to see.

Slide 08 Step 3 · Record first, decide later ≈40 s

That gave us step three: record first, decide later. FFmpeg never stops. It copies the main stream, without re-encoding, into 2-second segments in a 60-second ring in RAM. When an event ends, we join the segments from 4 seconds before the first motion to 5 seconds after the last one.

We tested it with manual triggers against the clock, and the first run found a bug: segments were timed when we polled them, so the timing drifted and the ends of clips were cut. We switched to the files' write times, added a test, and the ends were right. ***[Point to the clip]*** In this real clip the lizard is pulled at 5.1 seconds, and you can still see it before it moved. So far 97 clips have been checked and moved to the Mac.

Slide 09 Step 4 · First detector, first surprise

≈35 s

Step four was the first detector. A second FFmpeg process shrinks the substream to 160 by 90 and compares each frame with the one before. At home, with a hand, it worked: four events, each with 3.7 to 4.65 seconds before the hand appeared.

Then we ran it at school for two and a half hours. It saved 86 recordings — and there was no gecko in any of them. People walking, hands, lights and auto-exposure all counted as motion.

Slide 10 Step 5 · Tell it where to look

≈35 s

Step five was to tell it where to look. We draw the enclosure on the web page, and every change outside it is ignored. If 30 percent or more of the whole picture changes at once, we treat it as a scene change, not motion. And motion has to last half a second.

The test: for about two minutes people moved in the background and nothing was recorded, and a 31.4 percent whole-picture change was ignored. ***[Point to the third frame]*** But a hand inside the box still recorded. We also found the opposite problem — the toy lizard was often missed — so we lowered the threshold from 25 to 15.

Slide 11 Step 6 · Trigger on the gecko itself

≈35 s

That led to the main idea, step six: trigger on the gecko itself, not on any change. For this we needed a tracker — we built it just before this step, and I'll show how it works in two slides.

Now the area rules only write to the log. A clip starts only when the locked gecko really moves: the whole box must shift at least 2 pixels within a second, with both edges moving the same way. A shadow that only stretches one edge does not count.

Slide 12 Step 6 · Three rounds in the lab

≈40 s

We tested this in three rounds, and every failure became a rule. Round one, at 17:47: three clips with only a hand. The hand made the lizard's pixels flicker, so we required the centre to really move, for half a second.

Round two, at 18:04: two hand-only clips. The hand's shadow joined the lizard and pulled the box, so we added the whole-box shift, an overlap check and a size limit. Round three, at 18:45: a hand stayed in the area for eight seconds, the old area rule fired on 87 frames, nothing was recorded — and all three real lizard clips were kept.

Slide 13 How the tracker finds the gecko

≈30 s

Here is the tracker we relied on. It learns a background from still frames, marks the pixels that differ from it, groups them into blobs, and locks on the one that moves.

In our first run the box covered only the tail. The reason was brightness: the green lizard is almost the same grey as the desk — 157 against 156 to 186. In the colour channels they are far apart, so we added colour. It runs in pure Python at about 8 milliseconds per frame, down from 63 before optimising.

Slide 14 Tracker results

≈25 s

With colour, the box covers the whole lizard in 5 of 6 sampled frames, and the lizard stayed still for 55 seconds without a false "lost" — the first version had called it lost after about 100 seconds of rest. One limit: when the lizard is pulled away fast, the box can stay on the empty spot for about 2 seconds.

Slide 15 Where we are now

≈35 s

So, step by step, we went from 86 recordings with no gecko to zero hand-only clips, and every clip keeps 4 to 6 seconds before the first motion.

What we have not done yet: a real gecko — every test used a toy lizard — night recording with the camera's infrared, and runs longer than a few hours. Next, we test in the real enclosure, run a night test, give the ROCK its own power supply for a 24-hour run, and link clips to thermal events — which brings us to the thermal sensor. ***[Hand over to the next speaker]***

If someone asks

Why not an ESP32-C3 for video?	It has no Ethernet, its memory cannot hold one 2560×1440 frame, and it cannot decode H.264. So the recorder runs on the ROCK 4C+.
Does it work with a real gecko?	Not tested yet. Every test used a toy lizard; the real enclosure is our first next step.
Are the images on slides 08, 10, 12 and 14 live?	No. They are replays: recorded clips fed back through the same detector and tracker, with boxes drawn on each frame.

Facts from project-camera/experiments (30 Oct and 5 Nov 2026). Numbers match the Figma “Final PPT” slides 05–15.